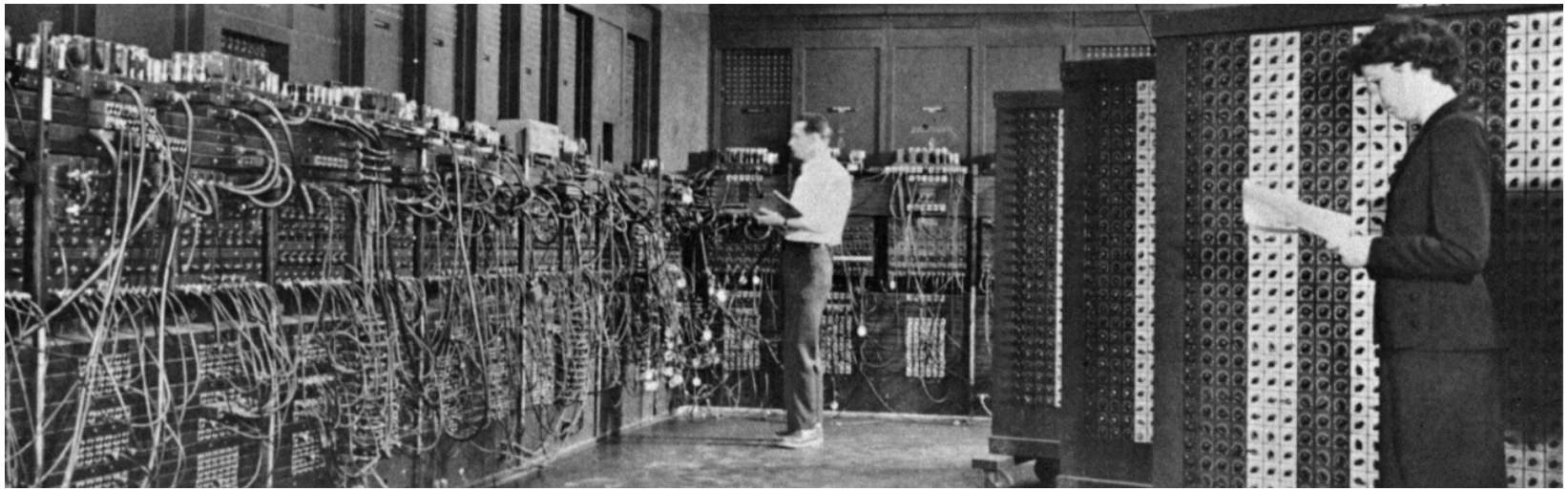
DREW

Lecture 1: Intro & Data Representation

CSCI 260 Computer Systems & Architecture
Fall 2024

Andy Siyuan Feng

08/26/2024



US Army Photo, University of Pennsylvania, public domain
ENIAC (Electronic Numerical Integrator and Computer),

Overall: What is this course about?

- We take a *bottom-up* approach to learn:
  - what *computers are consisted of;
  - how do they work;
  - how they are organized internally;
  - what are the design trade-offs and limitations;
  - ultimately, how are these constraints affecting programming and applications; in terms of efficiency, scalability, and security.

*Modern *general purpose* computers; not super /quantum computers.

- (Un)Fortunately,:
  NOT about how to fix a computer or building/buying the best one.
- Some ideas/guidelines though.

Overall: What is this course about?

- We take a *bottom-up* approach to learn:
  - Bits
  - Gates
  - Processor
  - Instructions
  - C Programming



Electronic-level

System-level

System/App.-level

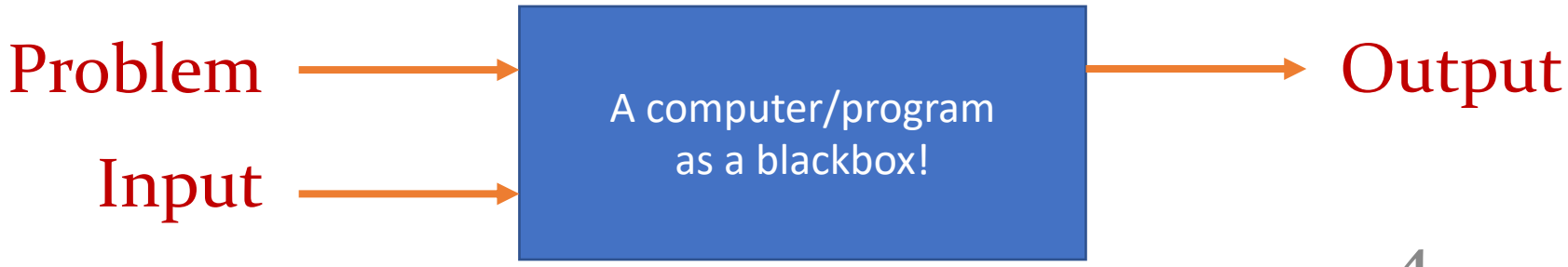
Low-level

High-level

- **End Goal**: Understanding what's going on underneath; be able to write C programs from an ISA-perspective and use UNIX command line tools for problem solving in advanced courses/profession.

Overall: What is this course about?

- **Abstraction**: One of the most important tool/mindset in CS.
- Bottom-up means that, we will start from the most fundamental building blocks, then gradually come to what we know as a computer.
- Once done with something: ignore its detail, put it in a *blackbox*, and treat it as a building block for the next level, and scale up.
  - Until the scale up causes a problem!
  - Hence, need to understand how components interact before moving on to the next level.

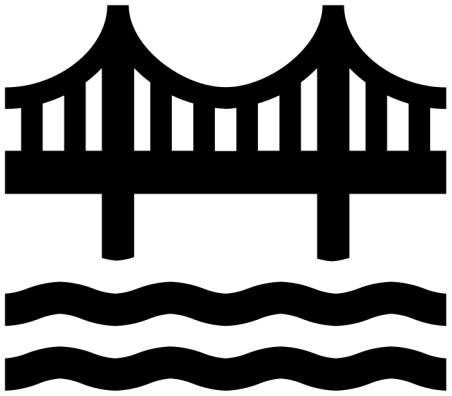


DREW

Overall: What is this course about?

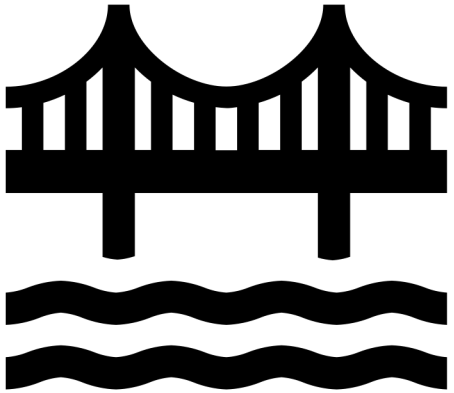
- **Software and Hardware** (including Operating Systems): Both are significant components of a computer system (Historically proven).
  - Thus, again, no matter which area to specialize in, want to know all three to some degree.
  - Helps us understand the capability and limitations of computing in general.

Hardware (Architecture)



Organization

General Purpose Computer

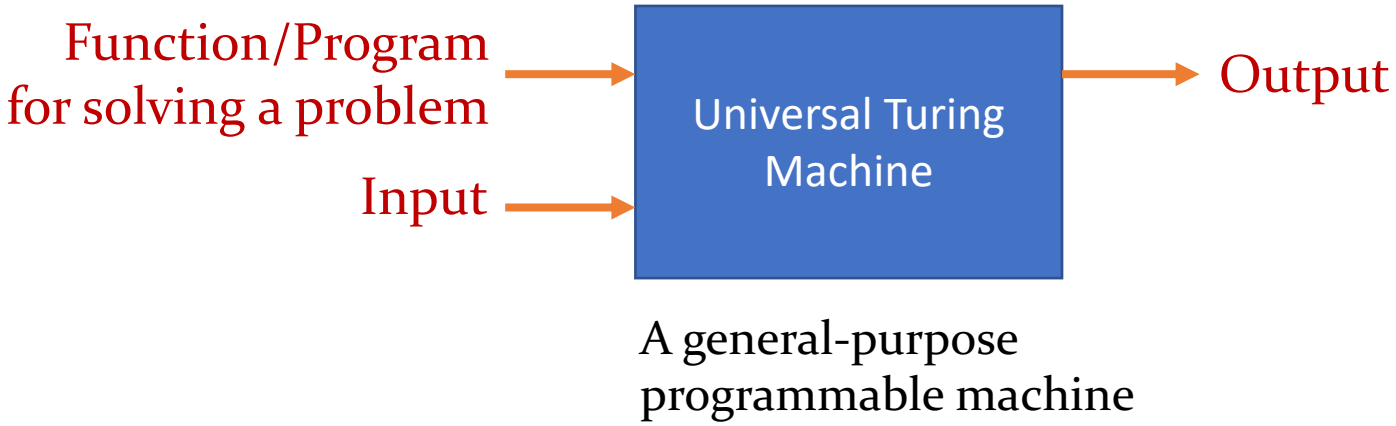


OS

Application Software

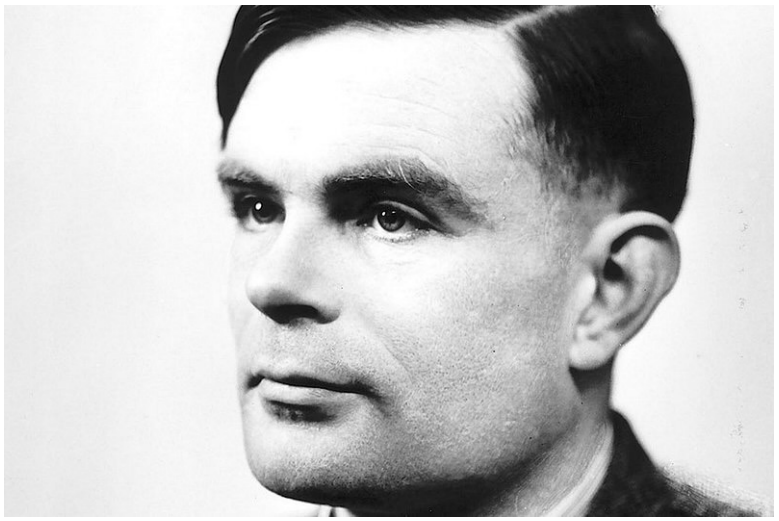
Overall: Two Very Important Ideas

1. **Universal Computational Devices**: Given enough physical resources and time, all computers are capable of computing the same things that are computable.
2. **Problem Transformation**: The ultimate objective of computation is to transform a problem expressed in natural language into one that can be solved by an electronic computer.



A general-purpose
programmable machine

Alan Turing, public domain



Overall: Two Very Important Ideas

- **The Church-Turing Thesis**: "Any real-world computation can be translated into an equivalent computation involving a Turing Machine." [Wolfram MathWorld]

- A problem-solving procedure in a computer:

  Problem → Algorithm → (language) → Program with Data Structure → (Compiling) →

  Instruction Set Architecture (***ISA***)

  [The Intel x86 ISA vs The Apple Silicon with ARM ISA]

  → Microarchitecture → Logic Circuit → Devices

Overall: What is this course about?

- **Bits and Bytes**: How do we represent info. from electrical signals?
- **Digital Logic**: How do build circuit to process the bit info.?
- **Processor and Instruction Set**: How do we build a process using logical elements? What functions does the processor have?
- **Assembly Language**: How to implement algorithms using these functions? How can code be reused?
- **I/O, Traps, Interrupts**: How does the processor communication with outside?
- **C Programming**: How to write in C and implement high-level structures?

Outline

- **Lecture 1, 2**: Number Representation: §2.1-§2.4
- **Lecture 2, 3, 4**: Integer Operations and Floating-Point Type: §2.5-§2.7
- **Labor Day (M)**: No class
- **Lecture 4 (W)**: Text Types: §2.7
- **Lecture 5**: In-class Exercise with attendance
  - HW1 due

§2: Bits, Data Type, and Operations

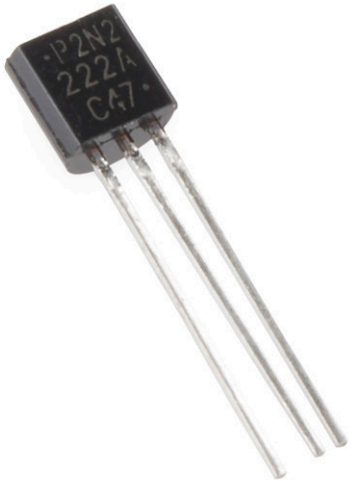
- **Question**: We, as humans, store data of different formats in computers, text, images, sound, videos, etc., that only WE recognize.
  - However, from a machine perspective, how are these data stored and in what sort of fashion?
  - How do these digital entities connect with the real world and its physical laws?

- Remember that a computer, at its lowest level, is an electronic machine, and it works by manipulating the flow of the electrons.

Transistors

- The lowest level component of a computer is called a *transistor*.
  - Made of semiconductors, such as Silicon, for controlling electric flows.
  - The invention led to the 1956 Nobel Prize in Physics.

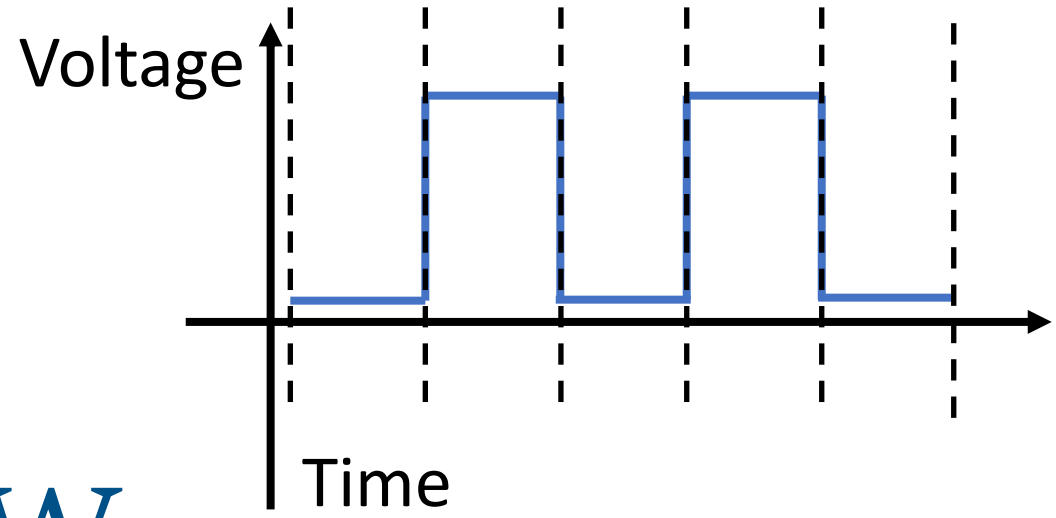
- When a transistor recognizes:
  - *the "presence" of a voltage* → we call this the state "1";
  - *the "absence" of a voltage* → we call this the state "0".
  - Here, it could mean high/low voltages.



Transistors

- When a transistor recognizes:
  - *the "presence" of a voltage* → we call this the state "1";
  - *the "absence" of a voltage* → we call this the state "0".

- Now, then, a 01010 sequence is recognized.



This is the *Binary* (2-ary) */Base 2 Representation* system in computers.

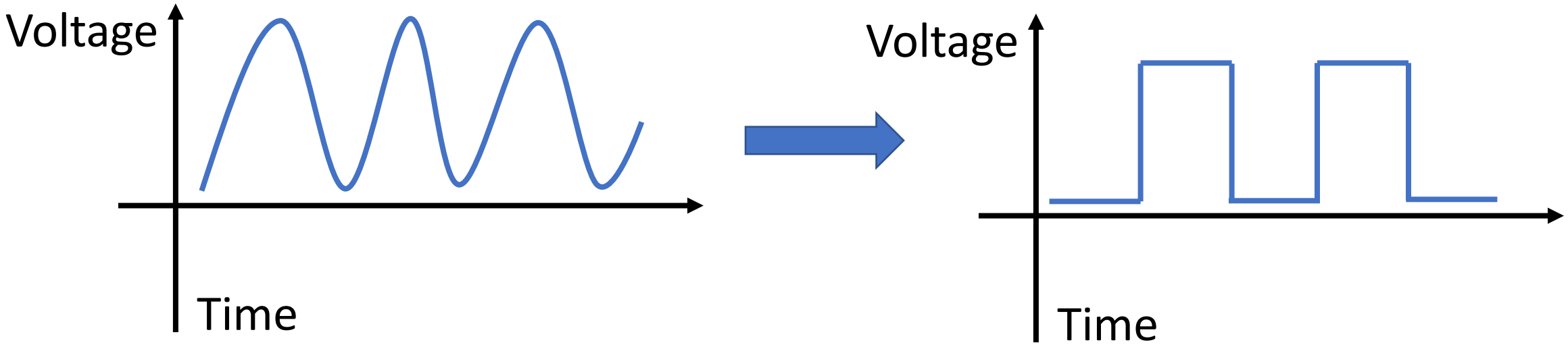
DREW

Transistors

- Discretization and Quantification: out of our scope here.

Real-world Analog signal
continuous

Digital signal
discrete



DREW

Binary System

- Binary sequence 01010:
  - Basic unit of information is the *binary digit*, or *bit*.
  - 01010 has 5 bits.

- **Bits and States in the computer**
  - A single bit can represent 2 states; i.e., state "0" and state "1".
  - In order to represent more states, need to employ more bits.

  - For 2 bits, possible sequences: 00, 01, 10, 11; i.e., 4 states.
  - For 3 bits, sequences: 000, 001, 010, 011, 100, 101, 110, 111; 8 states.
  - In general, for k bits, there will be $n = 2^k$ states.

A computer microchip nowadays
have at least tens of billions of transistors.

Binary System and Data

- What kind of data can be **represented**?
- *Data types* can be represented as binary:
  - **Numbers**: *unsigned*, signed, integers, floating point, etc.
  - **Text**
  - Images: pixels, colors, shapes, …
  - Sound: frequencies, time, …
  - Videos
  - Logical: T/F
  - **Instructions**
  - Etc.

A data type also needs to specify on how do data interact: **Operations**

Data type includes **both** Representation and Operations.

DREW

Unsigned Binary Integers

Before getting into binary numbers, something familiar,

- **Decimal** (Base 10) numbers with *weighted-positional notation*:
  - **Ex.**: In the decimal number 329, due to the positions of the digits,
    "3" is worth of 300, *the leading/most significant digit*
    "2" is worth of 20, and
    "9" is worth of just 9. *the least significant digit*
  - Hence, the *base 10 value of* 329 is calculated as $329 = 300 + 20 + 9$, or

$$329 = 3\times(10)^2 + 2\times(10)^1 + 9\times(10)^0.$$

- Notice how the exponents acts on the base relative to the corresponding positions of the digits.

Recall $10^0 = 1$.

DREW

Unsigned Binary Integers

- **Ex.**: Similarly, the decimal number 4752 has the decimal value of
$$4752 = 4\times10^3 + 7\times10^2 + 5\times10^1 + 2\times10^0.$$

- Now, let's apply the same idea onto binary numbers:
- **Ex.**: the binary number 101, or $(101)_2$, then has the decimal value of
$$1\times(2)^2 + 0\times(2)^1 + 1\times(2)^0 = 1\times4 + 0\times2 + 1\times1 = 5,$$
m.s.b. l.s.b.

  where the base is 2, and we apply the corresponding exponents relative to the positions of the digits.

- Hence, $(101)_2 = (5)_{10}$.

Most Significant Bit: m.s.b/MSB

Least Significant Bit: l.s.b/LSB

DREW

Unsigned Binary Integers

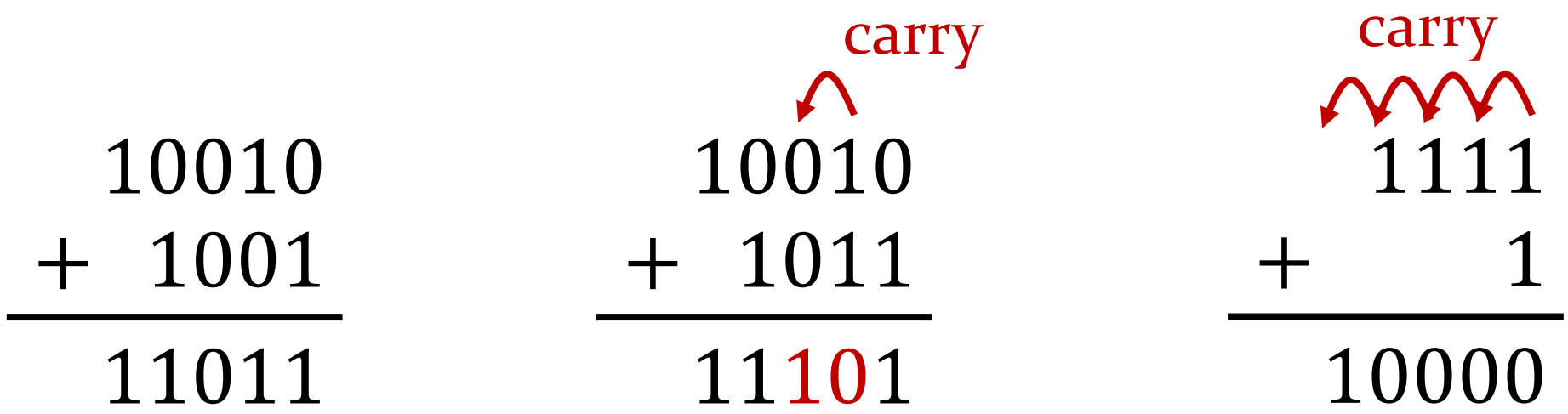
Why is that true? Consider the following enumeration starting from the 0th index.

- An k-bit unsigned integer can represents $n = 2^k$ values, ranging from 0 to $n - 1$.

2^2	2^1	2^0	Enumeration
0	0	0	$(0)_{10}$
0	0	1	$(1)_{10}$
0	1	0	$(2)_{10}$
0	1	1	$(3)_{10}$
1	0	0	$(4)_{10}$
1	0	1	$(5)_{10}$
1	1	0	$(6)_{10}$
1	1	1	$(7)_{10}$

Unsigned Binary Addition

- **Base 2 Addition**: again, just like in base 10.



- Also have subtraction, multiplication, and division. However, need more data types to define.

Signed Binary Integers

Like in base 10, only deal with the positive portion (and zero) won't be enough for all types of tasks.

- **Idea**: a k-bit binary sequence can represent 2^k distinctive values.
  - Assign about half of those values to positive integers;
  - and about half to negative integers;
  - then, we have two values left: one to the number zero, and one extra (will come back to this);
  - and use the m.s.b. to indicate the sign of the value.

Signed Binary Integers

- For a k-bit binary sequence
- **Positive Integers**: ranging from 1 to $2^{k-1} - 1$, since the m.s.b. is now used for the sign.
  - Just like unsigned binary: $00101 = +5$ and $01111 = +15$.
- **Negative Integers**: ranging from $-(2^{k-1} - 1)$ to -1.
  - The m.s.b to 1 to indicate the minus sign. However,
  - 3 ways (data types) to compute the value of the rest of the bits.
    1. Signed Magnitude
    2. Ones' Complement
    3. Two's Complement

Signed Magnitude – a naïve attempt **NOT Ideal**

- For a k-bit binary integer, the value of the integer is computed as

$$v_{10} = (b_k b_{k-1} \ldots b_1 b_0)_2 = (-1)^{b_k} \times \left(b_0 \cdot 2^0 + b_1 \cdot 2^1 + \cdots + b_{k-1} \cdot 2^{k-1}\right);$$

i.e., treat all the bits other than m.s.b as usual.

Ranges from $-(2^{k-1} - 1)$ to $2^{k-1} - 1$.

- **However**, two issues!

1. Issues with arithmetic:

   Consider $(10001)_2 + (00001)_2 = (10010)_2$, i.e., $-1 + 1 = -2$!!

2. There are two zeros: both 10000 and 00000; i.e., -0 and $+0$!!

Ones' Complement – a historic attempt **NOT Ideal**

- A complement of a bit is similar to the complement of a truth value
  - $\neg T = F$ and $\neg F = T$, similarly, $\neg 0 = 1$ and $\neg 1 = 0$.
- **Idea**: Simply flip all the bits in a positive number, so that they have complementing m.s.b.
  - **Ex.**: Since $(4)_{10} = (00100)_2$, $(-4)_{10} = (11011)_2$.

    Ranges from $-(2^{k-1} - 1)$ to $2^{k-1} - 1$.
- Again, there are two zeros, 00000 and 11111.
- In fact, it requires additional hardware implementation for arithmetic in One's Complement.

Two's Complement – finally

- **Idea**: First, flip all the bits in a positive number (1's Comp.), and then add 1 to the result.
  - **Ex.**: Since $(4)_{10} = (00100)_2$, $(-4)_{10} = (11011 + 1)_2 = (11100)_2$.

    Ranges from -2^{k-1} to $2^{k-1} - 1$.

- Now, there's a unique $0 = (00000)_2$; by doing 2' complement, we acquire $(11111 + 1)_2 = (\cancel{1}00000)_2$ again. (Called a "carry out".)

- **Advantages**: made operations easy, and efficient use of all bits.

Bin.	00000	00001	00010	00011	00100	00101	00110	00111	…
Dec.	0	1	2	3	4	5	6	7	…
Bin.	00000	11111	11110	11101	11100	11011	11010	11001	…
Dec.	0	-1	-2	-3	-4	-5	-6	-7	…

Two's Complement

- **A Shortcut**:

1. Copy bits from right to left until an "1" bit is seen.
2. Copy down the "1" bit.
3. From now on, flip and copy down all the bits to the left.

- **Ex.**: Take the two's complement of the signed binary number below.

```
011010000
    10000
 1001
----------
100110000   (2 Comp.)
```

```
011010000
100101111   (1's Comp.)
      +         1
----------------------
100110000
```

DREW

To Compute Two's Complement

- **Architecture**: CPU vs ALU

- Not everything is happening in the Central Processing Unit (CPU)
- Instead, the Arithmetic Logical Unit (ALU) is the dedicated circuit that handles arithmetic and logical on binary integers.

- Will come back.

Converting from 2's Comp. Binary to Decimal

- To convert a 2's comp. binary number to its decimal value:

1. Look at the leading bit to determine the sign.
2. If the leading bit is 0, then apply the exponent equation.
3. Otherwise, first apply 2's comp., and apply the exponent equation and add a minus sign to the obtained value.

- **Ex.**: Given 8-bit 2's comp. number $x = (11101000)_2$, compute its decimal value.
  - x is negative; hence, go to step 3.
  - x's 2's comp. is $y = 00011000$, where $(x)_{10} = -(y)_{10}$.
  - $(y)_{10} = 2^3 + 2^4 = 8 + 16 = 24$
  - $(x)_{10} = -(y)_{10} = -24$

It is recommended to get familiar with 2^0~2^{10}.

DREW

Converting from 2's Comp. Binary to Decimal

Do it after class

- **Ex.**: Given 8-bit 2's comp. number $x = (00100111)_2$, compute its decimal value. (Ans = $(39)_{10}$)

- **Ex.**: Given 8-bit 2's comp. number $x = (11100110)_2$, compute its decimal value. (Ans = $(-26)_{10}$)

- **Ex.**: Given 8-bit 2's comp. number $x = (01101000)_2$, compute its decimal value. (Ans = $(104)_{10}$)

Converting from Decimal to 2's Comp. Binary

- To convert a decimal number to its 2's comp. binary form:

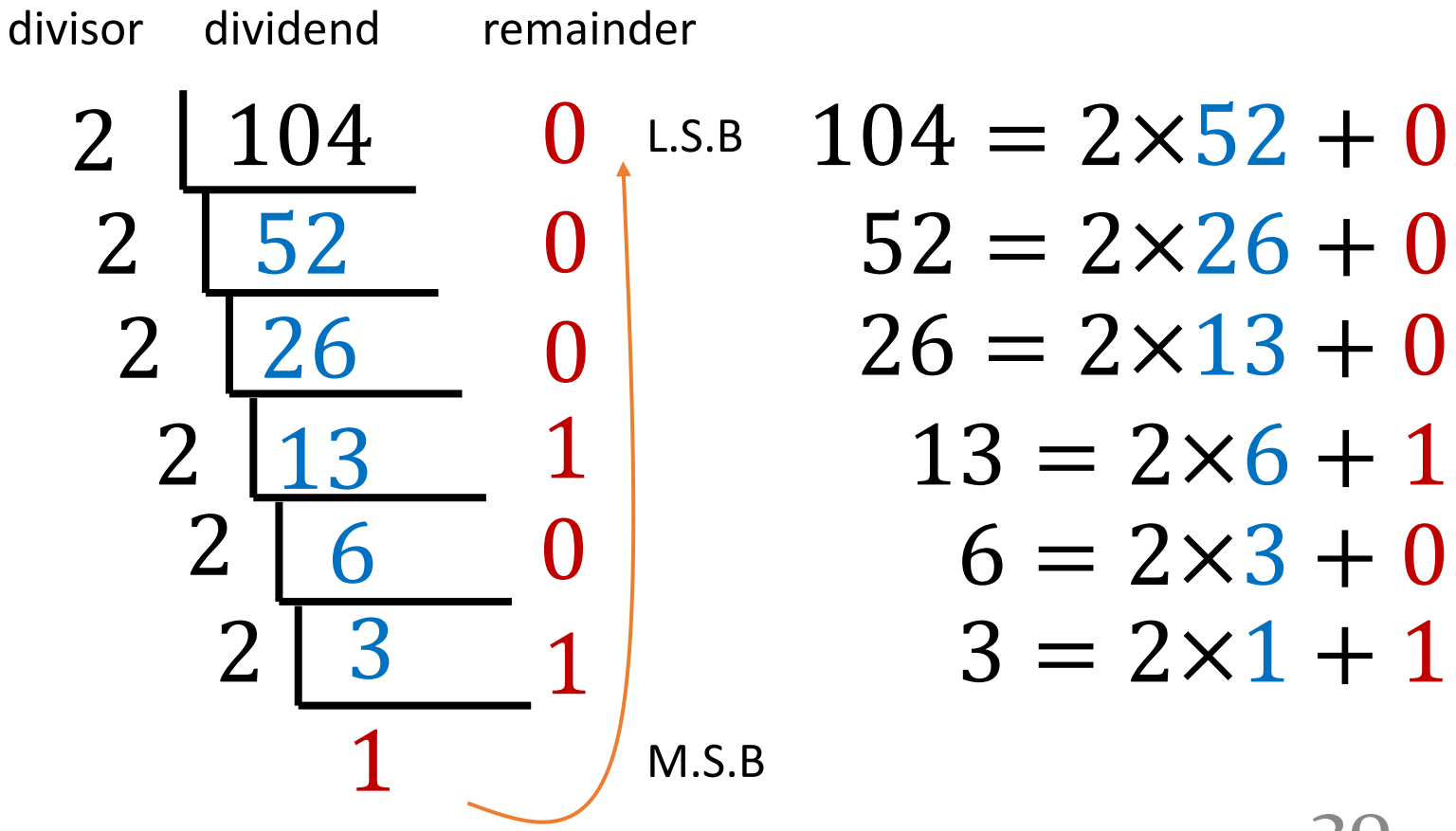
- **Method 1**: Short Division for Remainders

1. Take the magnitude of the decimal number (i.e., disregard sign) as the dividend.
2. Divide the current value by 2 and find the quotient and remainder.
3. Append the remainder from right to left, and let the quotient be the dividend.
4. If the dividend is 1, then record 1 as the m.s.b. Otherwise, go to Step 2.
5. Append at least one zeros to the left as needed.
6. If the original number was negative, take 2's complement.

Converting from Decimal to 2's Comp. Binary

- To convert a decimal number to its 2's comp. binary form:
- **Ex.**: Convert $(104)_{10}$ to its 8-bit 2's comp. binary form.

- **Ans**: $(01101000)_2$



DREW

Converting from Decimal to 2's Comp. Binary

- To convert a decimal number to its 2's comp. binary form: (bit different from the textbook, but equivalent)

- **Method 2**: Subtracting Powers of Two (the reversal of the exp. eqn.)

1. Take the magnitude of the decimal number.
2. Subtract largest power of two less than or equal to the number.
3. Put a one in the corresponding bit position for that power.
4. Keep performing the process until the result becomes zero.
5. Append at least one zeros to the left as needed.
6. If the original number was negative, take 2's complement.

Converting from Decimal to 2's Comp. Binary

- To convert a decimal number to its 2's comp. binary form:
- **Ex.**: Convert $(104)_{10}$ to its 8-bit 2's comp. binary form.

$$104 - 64 = 40 \qquad 64 = 2^6$$
$$40 - 32 = 8 \qquad 32 = 2^5$$
$$8 - 8 = 0 \qquad 8 = 2^3$$

- **Ans**: $(01101000)_2$

k	2^k
0	1
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1024

Converting from 2's Comp. Binary to Decimal

Do it after class

- **Ex.**: Given decimal number $x = (39)_{10}$, compute its 10-bit 2's comp. binary representation. (Ans =$(0000100111)_2$)

- **Ex.**: Given decimal number $x = (-26)_{10}$, compute its 10-bit 2's comp. binary representation. (Ans = $(1111100110)_2$)

A Generalization

- Recall that for a decimal integer with n digits, its value is computed as,

$$x = d_{n-1} \cdot 10^{n-1} + \cdots + d_1 \cdot 10^1 + d_0 \cdot 10^0$$

  where each digit d_i can take value a from 0 to 9.

- As well as, for a binary integer with n digits, its decimal value is,

$$y = d_{n-1} \cdot 2^{n-1} + \cdots + d_1 \cdot 2^1 + d_0 \cdot 2^0$$

  where each digit d_i can take value either 0 or 1.

- For a k-ary integer (i.e., base k) with n digits, its decimal value is,

$$v = d_{n-1} \cdot b^{n-1} + \cdots + d_1 \cdot b^1 + d_0 \cdot b^0$$

  where each digit d_i can take value from 0 to $b - 1$.

Extending to Numbers with Fractional Parts

- For a binary integer with n digits, its decimal value is,

$$v = d_{n-1} \cdot 2^{n-1} + \cdots + d_1 \cdot 2^1 + d_0 \cdot 2^0$$

where each digit d_i can take value from 0 to $b - 1$.

Fixed-Point Fractions: a "*binary point*" that's similar to a "*decimal point*"

- Notice the decreasing exponents act on the basis. Now, if we are to include fractional parts using the same approach, then

$$0.d_{-1}d_{-2}d_{-3} \ldots d_{-m}$$

where each digit d_i can take value from 0 to $b - 1$, will have a decimal value of

$$z = d_{-1} \cdot 2^{-1} + d_{-2} \cdot 2^{-2} + \cdots + d_{-m} \cdot 2^{-m}.$$

Extending to Numbers with Fractional Parts

- The binary number $0.d_{-1}d_{-2}d_{-3}\ldots d_{-m}$ has a decimal value of

$$z = d_{-1}\cdot 2^{-1} + d_{-2}\cdot 2^{-2} + \cdots + d_{-m}\cdot 2^{-m}$$
$$= d_{-1}\cdot 0.5 + d_{-2}\cdot 0.25 + d_{-3}\cdot 0.0125 + d_4\cdot 0.00625 + \cdots$$

Converting from Binary to Decimal

- **Ex.** (Bin. to Dec.): $(0.1011)_2 = 1\cdot 2^{-1} + 0\cdot 2^{-2} + 1\cdot 2^{-3} + 1\cdot 2^{-4}$
$= 0.5 + 0.0125 + 0.00625 = (0.6875)_{10}.$

- **Ex.** (Bin. to Dec.): $(0.01011)_2 =$

Extending to Numbers with Fractional Parts

Converting from Binary to Decimal

- Similarly, we keep subtracting the largest power of 2 less than or equal to the number while keep recording the corresponding digit values.
- **Ex.** (Dec. to Bin.): $(0.25625)_{10} = (??)_2$

1. Now, 0.5 is strictly greater than the number.
2. So, we take the next largest 2's power, 0.25 away from the number:
$$0.25625 - 2^{-2} = 0.00625$$
3. Now, proceed to the next largest 2's power, 0.125, which is too big.
4. So, we take the next, 0.00625, which is exactly what's left.

- Therefore, $(0.25625)_{10} = (0.0101)_2$.

Extending to Numbers with Fractional Parts

However, there's one issue **Ex.** (Dec. to Bin.): $(0.421)_{10} = (??)_2$

1. Firstly, 0.5 is strictly larger than the number.
2. Then, take 0.25 instead, $0.421 - 2^{-2} = 0.171$.
3. Then, take 0.125, $0.171 - 2^{-3} = 0.046$.
4. …

- We can keep up with the steps indefinitely, until what' s left is zero.
- However, NOT all cases (such as ours) can be expressed with a definite form!
- What can be done is to *approximate* by taking finitely many digits.
  E.g., $(0.421)_{10} = (0.01101)_2$ by taking 4 digits.

More digits, more accurate.

Extending to Numbers with Fractional Parts

Now, let's combine both parts:

- **Ex.**: $(39.6875)_{10} = (32 + 4 + 2 + 1 \ . \ 0.5 + 0.0125 + 0.00625)_2$
$= 100111.1011$

- Since the binary point always separate the positions holding 2^0 and 2^{-1}, this is the **Fixed Point** number.

- Will come back to nums with fractional parts with a different data type, **Floating Point** numbers.

Operations

- **Recall**: a data type includes both **representation** and **operations**.
- We now have a good representation for *signed integer*.
- Let's now look at its operations.

Arithmetic	**Logical**
Addition/Subtraction	Bitwise AND/OR/NOT
Sign Extension	Bitwise XOR
Shift/Multiplication	

- Plus, the issue of overflow.

Questions?

- **Next Lecture**:
  - Will be looking at more operations.
  - Number Data Type: Floating Point

- **HW 1** due on 09/06 (Fri.) by 11:59pm